FitPulse

Software Requirements

Specification

CSE494

Intelligent NoSQL Databases

Yash Kumar

12314638

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a complete, definitive, and technically rigorous description of the requirements for the web-based software application FitPulse — Your Personal Fitness Intelligence Platform. This document specifies the functional capabilities, non-functional constraints, external interface behaviors, architectural parameters, and database schemas necessary for software engineers, database architects, and quality assurance personnel to design, implement, verify, and maintain the system.

Intended Audience:

This document is prepared for:

1. Software Engineers & Developers: To guide implementation across the frontend (React, Vite, TypeScript, Tailwind CSS) and backend (Node.js, Express.js, TypeScript).

2. Database Architects & Evaluators: To review the MongoDB document schemas, compound indexing strategies, and multi-stage aggregation pipelines.

3. Academic Examiners & University Evaluators: To verify conformance with core computer science competencies in full-stack architecture, relational document modeling, and database-level data processing.

4. Quality Assurance (QA) Engineers: To derive unit, integration, end-to-end, and security test suites.

5. Project Stakeholders & Maintainers: To establish baseline system boundaries and plan future third-party biometric integrations.

1.2 Scope

1.2.1 Product Identification

The software product specified herein is designated as FitPulse — Your Personal Fitness Intelligence Platform (internal project identifier: fitpulse-monorepo, encompassing fitpulse-backend and fitpulse-frontend).

1.2.2 System Capabilities (What the Software Will Do)

FitPulse is an end-to-end full-stack MERN (MongoDB, Express.js, React, Node.js) web application engineered to collect, aggregate, visualize, and correlate multi-domain biometric and lifestyle logs into actionable health intelligence. The software will:

- User Identity & Profile Management: Provide token-based authentication (JWT via secure HttpOnly cookies and Bearer headers), password hashing via bcrypt, and user profile customization (height, weight, activity baseline, dietary/sleep goals, metric/imperial preferences).

- Activity & Step Tracking: Facilitate manual entry, editing, and deletion of daily step counts; automatically compute walking distance via height-proportional stride modeling (Stride = Height * 0.414 / 100 meters) and walking caloric expenditure; and track continuous adherence streaks.

- Nutrition & Calorie Intelligence: Allow itemized logging across five designated meal categories (Breakfast, Lunch, Dinner, Snack, Drink) using an embedded 45+ item catalog of verified staples (specifically calibrated for Indian cuisine, including Roti, Dal, Paneer, Rice, Rajma, and Chole) and custom food creation. Preserve historical nutritional integrity by embedding macro snapshots at log time.

- Workout & Overload Tracking: Record multi-set resistance training sessions (exercise name, set number, repetitions, weight in kilograms, rest intervals) and cardiovascular exercises (duration, type, MET-derived caloric burn); calculate cumulative session volume (Volume = Sum of reps * weight); and store reusable workout templates.

- Sleep Architecture Monitoring: Record bedtime and wake-up timestamps using ISO-8601 UTC date-times, accurately resolving durations that cross midnight; collect 1–5 self-reported quality scores; and compute schedule consistency indices.

- Granular Hydration Logging: Track water intake in milliliters with single-tap increments (100 ml, 250 ml, 500 ml) and custom volume logging, backed by daily goal tracking.

- Anthropometric Progress: Record body weight and circumferences (waist, hip, chest, arm, thigh) over time, automatically synchronize profile baselines, and calculate historical progress deltas.

- Goal Management: Manage active and historical targets across all seven health domains with progress percentage calculation.

- Real-Time Analytics via MongoDB Aggregations: Execute eight multi-stage MongoDB aggregation pipelines on the database engine (avoiding client-side or in-memory JavaScript array filtering) to produce weekly step cadence, macro distributions, training volume rankings, cross-collection caloric balance joins, monthly progress trends, an anonymized leaderboard, and a 5-pillar composite Lifestyle Consistency Score (0–100).

1.2.3 System Limitations & Exclusions (What the Software Will Not Do)

To ensure transparency and preserve data integrity, the initial release explicitly will not:

- Hardware Sensor Spoofing: The web application will not claim or pretend to read hardware pedometers directly through a desktop browser. Step entries are entered manually by the user or through mock-sync endpoints.

- Clinical or Medical Diagnosis: The application will not diagnose eating disorders, metabolic conditions, sleep apnea, or medical risks. All caloric balances, MET calculations, and the Lifestyle Consistency Score are habit-tracking metrics and are accompanied by non-clinical disclaimers.

- Automated Optical Recognition: The application will not perform computer-vision-based food image recognition; food logging is catalog- and parameter-driven.

- Mandatory Demographics: The application does not collect gender as a mandatory operational field, ensuring privacy and non-discriminatory metric modeling.

1.2.4 Benefits, Objectives, and Technical Goals

- Elimination of Monolithic In-Memory Processing: All analytics operations are offloaded to MongoDB aggregation stages ($match, $group, $unwind, $lookup, $project, $facet), guaranteeing sub-150ms query turnaround times across 30-day date ranges.

- Multi-Tenant User Isolation: 100% of read, write, update, and deletion requests enforce authorization against the authenticated user’s ObjectId, preventing horizontal privilege escalation.

- Zero Mock / Hardcoded State: Every dashboard card, chart, and progress ring is populated through RESTful endpoints communicating directly with the underlying MongoDB persistence layer.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym | Definition

SRS | Software Requirements Specification; the formal specification document.

MERN | Technology stack comprising MongoDB, Express.js, React.js, and Node.js.

ODM | Object Document Mapper; specifically Mongoose, which translates between Node objects and MongoDB BSON.

JWT | JSON Web Token; an RFC 7519 open standard for securely transmitting cryptographically signed claims between client and server.

HttpOnly Cookie | A secure cookie attribute that prevents client-side scripts from accessing session tokens, mitigating Cross-Site Scripting (XSS) threats.

BSON | Binary JSON; the binary serialization format used by MongoDB to store documents and execute pipelines.

Aggregation Pipeline | A MongoDB framework for data processing modeled on data-flow concepts where multi-stage operators transform documents into aggregated results.

Cross-Collection Join | The joining of distinct collection datasets via the $lookup aggregation operator (used in Pipeline 5 for calorie balance analysis).

MET | Metabolic Equivalent of Task; a physiological measure expressing the energy cost of physical activities relative to resting baseline.

Progressive Overload | The fitness training principle of gradually increasing weight, frequency, or repetitions to stimulate adaptation, tracked as volume (kg * reps).

stdDevPop | Population Standard Deviation; the MongoDB aggregation operator used to calculate sleep duration dispersion for schedule consistency scoring.

Zod | A TypeScript-first schema declaration and validation library with static type inference.

CORS | Cross-Origin Resource Sharing; a browser security mechanism that restricts HTTP requests initiated from scripts outside the origin domain.

KPI | Key Performance Indicator; critical numeric dashboard metrics (e.g., Daily Steps, Calorie Balance, Water Intake).

Idempotency | The property of certain database operations (such as the database seed script) whereby multiple identical executions yield the exact same result without duplicate accumulation.

1.4 References

1. IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications, IEEE Computer Society, 1998.

2. ISO/IEC/IEEE 29148:2018: Systems and software engineering — Life cycle processes — Requirements engineering, International Organization for Standardization, 2018.

3. MongoDB Aggregation Framework Documentation: Aggregation Operations and Pipeline Stages, MongoDB Inc., Manual v8.0, 2024. Available at: https://www.mongodb.com/docs/manual/aggregation/

4. RFC 7519: JSON Web Token (JWT), Internet Engineering Task Force (IETF), May 2015. Available at: https://datatracker.ietf.org/doc/html/rfc7519

5. Ainsworth BE, et al.: 2011 Compendium of Physical Activities: a second update of codes and MET values, Medicine & Science in Sports & Exercise, 2011 (used for MET caloric expenditure calculations).

6. National Institute of Nutrition (ICMR-NIN): Indian Food Composition Tables (IFCT), Hyderabad, India, 2017 (used for baseline nutritional and macronutrient profiles of Indian staples).

1.5 Overview

The remainder of this SRS document is structured as follows:

- Section 2 — Overall Description: Describes the product perspective, system interfaces, user characteristics, operating environment, design constraints, and general assumptions and dependencies.

- Section 3 — Specific Requirements: Details external interface requirements, software functional requirements categorized by functional module, system data structures, and the eight MongoDB aggregation pipeline specifications.

- Section 4 — Non-Functional Requirements: Details performance targets, security mechanisms, database integrity constraints, software quality attributes (maintainability, portability, reliability), and verification metrics.

- Section 5 — Database Design & Data Dictionary: Formally specifies the ten MongoDB collection schemas, embedded vs. referenced document designs, and indexing strategies.

- Section 6 — Verification & Testing Matrix: Outlines the automated integration test suite, testing coverage criteria, and demonstration walkthrough protocols.

2. General Description

This section describes the general context, operational factors, user environments, and technical boundaries that govern FitPulse — Your Personal Fitness Intelligence Platform. The information presented here contextualizes the specific technical requirements articulated in subsequent sections.

2.1 Product Perspective

FitPulse is a modern, self-contained, three-tier web application operating within the MERN (MongoDB, Express.js, React, Node.js) technological ecosystem. It is designed to function as an autonomous personal health intelligence platform, bridging the gap between basic spreadsheet-like trackers and enterprise fitness software.

1. System Architecture & Placement:

FitPulse operates in a classic client-server decoupled topology:

- Presentation Tier: A single-page application (SPA) built with React 18, Vite, TypeScript, Tailwind CSS, and Recharts. The client renders interactive visualization cards, progress rings, and tabular histories, operating strictly in modern web browsers.

- Application Tier: An asynchronous, event-driven RESTful API server built on Node.js and Express.js with TypeScript. This tier handles routing, input sanitization via Zod, cryptographic verification via JWT and bcrypt, business logic (MET calorie estimation, stride calculations), and request authorization.

- Database Tier: A document-oriented MongoDB database (version 8.x or MongoDB Atlas cluster) managed through Mongoose ODM. This tier houses ten discrete collections, compound indexing structures, and executes multi-stage aggregation pipelines directly on the storage cluster.

2. System Interfaces:

- Client to Server Interface: Standard HTTPS/HTTP REST API endpoints exchanging JSON payloads over port 5000 (or cloud gateway). Authentication state is passed via HttpOnly cookies and Bearer authorization tokens.

- Server to Database Interface: Native TCP connection string protocol (mongodb:// or mongodb+srv://) utilizing Mongoose for schema enforcement, auto-indexing, and aggregation execution.

- External Integrations (Future Architectural Context): Designed with modular abstraction layers capable of ingesting external wearable feeds (Apple HealthKit, Google Health Connect) through structured webhook and mock-sync endpoints without requiring schema rewrites.

3. User Interfaces:

- A responsive, dark-mode-first dashboard interface adhering to a high-contrast palette (background #0B0F19, surface #151B2B, cards #1C2435, electric lime accent #A3E635, violet accent #8B5CF6).

- Accessible navigation featuring a collapsible sidebar for widescreen desktops and an overlay drawer for mobile viewports.

- Integrated modal dialogs, accessible form validation indicators, and real-time toast feedback.

4. Hardware Interfaces:

FitPulse requires no specialized hardware on the client device. It executes within standard desktop, laptop, tablet, or smartphone web browsers that support modern ECMAScript standards and HTML5 Canvas/SVG rendering.

5. Software Interfaces:

- Client Runtime: Chrome 90+, Mozilla Firefox 88+, Safari 14+, Microsoft Edge 90+.

- Server Runtime: Node.js version 18.0.0 or higher (version 24.x tested).

- Database Engine: MongoDB Community Server 7.x/8.x or MongoDB Atlas M0/M10 clusters.

6. Communications Interfaces:

- Transport Layer Security (TLS 1.2 / 1.3) over TCP/IP for encrypted production communications.

- Cross-Origin Resource Sharing (CORS) rules restricting unauthorized domain origins while supporting localized university evaluation setups.

2.2 Product Functions

FitPulse provides nine core functional subsystems designed to track, calculate, and correlate lifestyle data:

1. Authentication & Security Engine:

- User registration, login verification, token generation, and secure cookie storage.

- Multi-tenant data authorization ensuring that records belonging to one user cannot be retrieved, edited, or deleted by any other user.

- Account profile modifications and cascading account deletion (deleting all associated logs across all ten collections upon user confirmation).

2. Activity & Pedometer Intelligence:

- Manual recording, updating, and removal of daily step counts.

- Automated stride calculation based on user height: Stride (km) = (Height in cm * 0.414) / 100,000.

- Walking caloric burn estimation based on body weight: Calories = Steps * (Weight in kg / 70) * 0.045.

- Activity streak tracking calculating consecutive days of logged activity.

3. Nutrition & Macronutrient Accounting:

- Structured logging categorized into Breakfast, Lunch, Dinner, Snacks, and Drinks.

- Catalog search against a verified 45+ item starter database (incorporating regional Indian staples such as Roti, Dal, Paneer, Rice, Chole, Rajma, Upma, Poha, and Dosa).

- Custom food creator permitting custom nutritional entries.

- Historical macro preservation embedding exact grams of protein, carbohydrate, fat, fiber, and calories into each meal log at insertion time.

4. Exercise & Workout Management:

- Multi-category logging: Strength Training, Cardio, Running, Cycling, Walking, Yoga, Pilates, Mobility, and Custom.

- Multi-set resistance logging capturing sets, repetitions, weights lifted (kg), and rest periods.

- Progressive overload volume calculation: Volume = Sum of (reps * weight in kg).

- Metabolic Equivalent of Task (MET) caloric expenditure computation: Calories = (MET * 3.5 * Weight in kg / 200) * Duration in minutes.

- Creation and deployment of reusable workout routines and templates.

5. Sleep Monitoring & Schedule Consistency:

- Logging of sleep start and wake-up times via ISO UTC timestamps.

- Mathematical duration resolution correctly spanning midnight boundaries without date corruption.

- Collection of self-reported 1–5 star sleep quality ratings.

- Statistical consistency calculation based on sleep duration dispersion across evaluated nights.

6. Hydration Management:

- Single-tap incremental logging (100 ml, 250 ml, 500 ml) and custom volume logging.

- Real-time comparison against daily hydration targets with dynamic visual progress indicators.

7. Anthropometric Tracking:

- Longitudinal logging of body weight and body circumferences (waist, hip, chest, arm, thigh).

- Automatic synchronization of the user profile baseline weight with the latest measurement entry.

- Historical progress delta computation (initial weight versus current weight).

8. Multi-Domain Goal Engine:

- Configurable target definitions across all seven health domains (Steps, Calories, Protein, Water, Sleep, Workouts, Body Weight).

- Dynamic percentage completion and active status toggling.

9. Advanced Aggregation Analytics:

- Execution of eight dedicated MongoDB aggregation pipelines for dashboard metrics, category distributions, training volume rankings, cross-collection caloric balance joins, monthly progress

summaries, anonymized community step rankings, and a 5-pillar composite Lifestyle Consistency Score (0–100).

2.3 User Characteristics

FitPulse accommodates three primary user categories:

1. Fitness Enthusiasts & Habit Trackers:

- Characteristics: Individuals seeking an integrated platform to monitor daily lifestyle habits (steps, calories, workouts, water, and sleep) in one unified dashboard.

- Technical Experience: Basic familiarity with consumer web applications and mobile forms.

- Cognitive Demands: Requires clean visual summaries, one-tap quick-add buttons (e.g. hydration quick-add), and clear habit indicators without overwhelming medical jargon.

2. Resistance & Strength Athletes:

- Characteristics: Users focused on progressive overload, lifting volume, and macronutrient targets (specifically daily protein thresholds).

- Technical Experience: Familiar with sets, repetitions, RPE/rest intervals, and nutritional tables.

- System Demands: Requires multi-set workout entry tables, exercise volume aggregation, and detailed macro breakdown charts.

3. Academic Evaluators & University Technical Reviewers:

- Characteristics: University professors, project examiners, and software engineering recruiters assessing the technical implementation, architectural rigor, and database design.

- Technical Experience: Advanced computer science knowledge; expertise in database schemas, indexing, aggregation logic, RESTful design, and automated test execution.

- System Demands: Requires verifiable database-level processing (no hardcoded state or in-memory array manipulation), clear demonstration credentials (one-click demo login), and comprehensive automated test suites.

2.4 General Constraints

The design and implementation of FitPulse are governed by several explicit technical and regulatory constraints:

1. Regulatory & Medical Constraints:

- Non-Clinical Status: FitPulse is explicitly classified as a habit-tracking and fitness-logging platform. It does not provide medical diagnoses, clinical calorie prescriptions, or medical risk analyses. All biometric algorithms are accompanied by non-clinical habit disclaimers.

- Voluntary Demographic Data: In alignment with modern data minimization standards, gender is not collected as a mandatory operational field.

2. Architectural & Technological Constraints:

- MERN Stack Mandate: The system must be implemented strictly within the MERN architecture (MongoDB, Express.js, React, Node.js) using TypeScript across both client and server tiers.

- Server-Side Aggregation Mandate: All analytics queries must execute via native MongoDB aggregation pipelines ($match, $group, $unwind, $lookup, $project, $facet). Filtering or grouping raw document arrays in client-side or Node.js runtime memory is prohibited for dashboard and analytics views.

- No Paid Third-Party API Dependencies: The system must function completely out-of-the-box using internal catalog data and local calculation engines without requiring paid API keys or external subscriptions.

3. Security & Data Protection Constraints:

- Password Security: Passwords must be hashed using bcrypt with a minimum work factor of 10 prior to database storage. Plaintext passwords must never be stored, logged, or returned in API responses.

- Transmission Security: Authentication tokens must be transmitted via secure, HttpOnly, SameSite cookies or cryptographically signed Authorization Bearer headers.

- Multi-Tenant Data Isolation: All database operations must enforce user authorization filters matching the authenticated token to prevent unauthorized record access.

- Denial of Service & Abuse Mitigation: Public API endpoints must be protected by IP-based rate limiting (500 requests per 15-minute window) and security headers via Helmet.

4. Platform & Environmental Constraints:

- Operating System Support: The local application and development scripts must be fully operable on Microsoft Windows (PowerShell compatible) as well as POSIX environments (macOS, Linux).

- Port Allocations: By default, the backend server binds to port 5000 and the Vite frontend binds to port 5173.

2.5 Assumptions and Dependencies

The functional requirements stated in this document are predicated on the following operational assumptions and technical dependencies:

1. Database Availability Assumption:

It is assumed that a MongoDB instance (version 7.0 or higher) is accessible via standard URI connection parameters (mongodb://127.0.0.1:27017/fitpulse or MongoDB Atlas cloud connection). If MongoDB is unavailable or network-partitioned, the server will log a fatal database error and terminate, as client-side localStorage fallback is intentionally prohibited to preserve academic authenticity.

2. Modern Browser Support Assumption:

It is assumed that client users access the web platform through a modern web browser equipped with a JavaScript engine supporting ES2020+, CSS Grid, Flexbox, and HTML5 SVG rendering. Users with legacy browsers (such as Internet Explorer) are assumed to be outside the supported user base.

3. Client Clock Synchronization Assumption:

It is assumed that client hardware devices maintain a reasonably synchronized system clock (+/- 5 minutes) via NTP (Network Time Protocol) to ensure that local date inputs and UTC ISO timestamps map accurately to calendar day boundaries.

4. User Truthfulness & Input Integrity Assumption:

Because FitPulse operates without invasive surveillance or biometric sensor mandates, the system assumes that step counts, meal portions, sleep hours, and water intake entered by the user are reasonably truthful approximations. The calculation models (e.g. MET calories, stride distances) are accurate within mathematical limits based on the user's provided baseline parameters (height and weight).

5. Third-Party Dependency Stability:

The software relies on established open-source libraries:

- Express.js (HTTP routing and middleware pipeline)

- Mongoose (ODM schema validation and aggregation builder)

- Zod (Runtime input validation)

- Recharts & Framer Motion (Data visualization and UI rendering)

- Tailwind CSS (Utility-first styling system)

Changes or deprecations in these core open-source packages may require corresponding updates to configuration or build scripts.

3. Specific Requirements

This section specifies the detailed functional, external interface, non-functional, and design requirements for FitPulse — Your Personal Fitness Intelligence Platform. Each requirement is stated unambiguously with a unique identifier, traceable functional logic, verifiable acceptance criteria, and priority rating (High, Medium, Low).

3.1 External Interface Requirements

3.1.1 User Interfaces

The FitPulse user interface is an accessible single-page web application implementing a high-contrast dark-mode fitness aesthetic (Background: #0B0F19, Surface: #151B2B, Card: #1C2435, Electric Lime: #A3E635, Violet: #8B5CF6).

UI-01 Navigation Layout:

- Desktop viewports (1024px and wider) shall render a collapsible persistent sidebar with icons, text labels, and an indicator of live database status.

- Mobile viewports (under 1024px) shall render an interactive hamburger toggle activating a sliding drawer overlay navigation.

UI-02 Visual Feedback & Notifications:

- Interactive actions (creation, editing, deletion) shall display non-blocking visual feedback and asynchronous loading indicators.

- Destructive actions (record deletion, account termination) shall trigger modal confirmation dialogs with cancel and confirm options.

UI-03 Responsive Visualizations:

- Charts generated via Recharts (bar, area, pie, and line graphs) and custom SVG progress rings shall resize dynamically across viewports ranging from 320px to 4K resolutions without clipping or label truncation.

3.1.2 Hardware Interfaces

FitPulse operates as a client-server web system and does not interface directly with physical sensors or proprietary device buses:

- Client Device: Standard consumer computing devices (desktops, laptops, tablets, smartphones) equipped with an input pointer, keyboard, or touchscreen display.

- Display Resolution: Minimum operational display resolution of 320 x 568 pixels; optimized for 1920 x 1080 pixels and common mobile viewports.

- Host Server: Standard cloud or physical server instance (minimum 1 vCPU, 1 GB RAM, 10 GB storage for development and testing).

3.1.3 Software Interfaces

SI-01 Client-Side Runtime:

- Compatible with Chromium-based browsers (Chrome 90+, Edge 90+, Brave), Mozilla Firefox 88+, and WebKit (Safari 14+).

- Requires ECMAScript 2020 (ES11) support and HTML5 Canvas / SVG rendering capability.

SI-02 Server-Side Runtime & Environment:

- Node.js runtime version 18.0.0 or higher (version 24.x supported).

- Express.js version 4.21+ application server.

SI-03 Database Management System:

- MongoDB Community Server version 7.0+ or 8.0+, or MongoDB Atlas cloud cluster.

- Mongoose ODM version 8.9+ managing BSON serialization, schema constraints, and pipeline aggregation execution.

SI-04 Development & Testing Tooling:

- TypeScript compiler (tsc) target ES2022.

- Jest test runner version 29.x with Supertest version 7.x.

3.1.4 Communications Interfaces

CI-01 Network Protocols:

- Client-server communication shall execute over HTTP/1.1 or HTTP/2. In production, transport shall be encrypted via TLS 1.2 or TLS 1.3 over HTTPS.

- Standard port allocations: Port 5000 for backend API, Port 5173 for Vite development server, Port 27017 for local MongoDB daemon.

CI-02 Data Serialization:

- All request and response bodies shall be formatted as standard application/json payloads encoded in UTF-8.

CI-03 Session Transport:

- Authenticated state shall be conveyed through HttpOnly, SameSite cookies named 'token' or via the HTTP Authorization header using the 'Bearer <jwt>' schema.

3.2 Functional Requirements

3.2.1 User Authentication and Profile Management Subsystem

3.2.1.1 Introduction

Governs user identity creation, session establishment, credential verification, profile baselines, and account lifecycle management.

3.2.1.2 Inputs

- Registration: Full name (string, 2–100 chars), email address (valid RFC 5322 format), password (string, minimum 6 chars), optional date of birth (ISO date), height in cm (number, 50–300), weight in kg (number, 20–500), activity level (enum: sedentary, lightly_active, moderately_active, very_active, extra_active), fitness goal (enum: muscle_gain, fat_loss, weight_maintenance, general_fitness, improved_endurance).

- Login: Email address and plaintext password.

- Profile Update: Partial profile fields, unit preferences (metric or imperial), and daily goals.

- Password Change: Current password, new password (minimum 6 chars).

3.2.1.3 Processing

- FR-AUTH-01 (Priority: High): The system shall validate registration inputs using Zod schemas. If the email is already registered, registration shall be rejected.

- FR-AUTH-02 (Priority: High): The system shall generate a cryptographic salt (work factor 10) and hash the user's password using bcrypt prior to database insertion.

- FR-AUTH-03 (Priority: High): Upon valid authentication, the system shall generate a signed JSON Web Token (JWT) with a 7-day expiration containing the user ID, email, and timezone.

- FR-AUTH-04 (Priority: High): The JWT shall be attached to the HTTP response as an HttpOnly cookie and also returned in the response body.

- FR-AUTH-05 (Priority: High): When an account deletion is requested, the system shall cascade-delete all records linked to the user's ID across all ten MongoDB collections.

3.2.1.4 Outputs

- HTTP 201 Created on registration with user profile object (excluding password hash) and token.

- HTTP 200 OK on login with user profile object and token.

- HTTP 200 OK on profile update and password change confirmation.

3.2.1.5 Error Handling

- Return HTTP 400 Bad Request with field-level error arrays if validation fails.

- Return HTTP 401 Unauthorized for invalid login credentials or expired tokens.

- Return HTTP 409 Conflict if registration email already exists.

3.2.2 Interactive Health Overview Dashboard Subsystem

3.2.2.1 Introduction

Aggregates and displays the six core Key Performance Indicator (KPI) cards and cross-domain visualizations for a selected calendar date.

3.2.2.2 Inputs

- Authenticated user ID (derived from JWT).

- Target date parameter (query string format YYYY-MM-DD; defaults to current date if omitted).

3.2.2.3 Processing

- FR-DASH-01 (Priority: High): The system shall concurrently query MongoDB collections using Promise.all to fetch: daily steps, daily meals, daily water logs, latest sleep record, 7-day workout history, and latest weight measurement.

- FR-DASH-02 (Priority: High): The system shall compute KPI Card 1 (Daily Steps): current steps, target steps, completion percentage, and estimated walking distance.

- FR-DASH-03 (Priority: High): The system shall compute KPI Card 2 (Calories): total calories consumed, total estimated calories burned (walking plus workout expenditure), intake target, and net caloric balance.

- FR-DASH-04 (Priority: High): The system shall compute KPI Card 3 (Hydration): total water consumed in ml, daily water target, and completion percentage.

- FR-DASH-05 (Priority: High): The system shall compute KPI Card 4 (Sleep): most recent sleep duration in hours and minutes, target sleep duration, and self-reported quality score (1–5).

- FR-DASH-06 (Priority: High): The system shall compute KPI Card 5 (Workouts): number of completed workouts in the past 7 days and total exercise minutes.

- FR-DASH-07 (Priority: High): The system shall compute KPI Card 6 (Nutrition): grams of protein consumed, protein target, carbohydrate grams, and fat grams.

- FR-DASH-08 (Priority: High): The system shall construct four visualization payloads: weekly step cadence array, calorie intake versus total burn comparison array, macronutrient calorie distribution array, and 14-day Lifestyle Consistency Score breakdown.

3.2.2.4 Outputs

- HTTP 200 OK containing JSON structure with user profile metadata, six KPI card objects, and visualization arrays.

3.2.2.5 Error Handling

- Return HTTP 401 Unauthorized if request lacks valid session token.

- Return HTTP 404 Not Found if authenticated user ID is not found in database.

3.2.3 Activity and Pedometer Tracking Subsystem

3.2.3.1 Introduction

Enables logging, updating, and historical review of daily step counts, walking distance, walking caloric expenditure, and streak continuity.

3.2.3.2 Inputs

- Date (string format YYYY-MM-DD).

- Steps (integer, 0 to 200,000).

- Optional distance in km and estimated calories burned.

- Source tag (enum: manual, sync, wearable; defaults to manual).

3.2.3.3 Processing

- FR-ACT-01 (Priority: High): If distance in km is not supplied, the system shall calculate it using height: Stride (km) = (Height in cm * 0.414) / 100,000. If height is unavailable, a standard 0.000762 km stride factor shall be applied.

- FR-ACT-02 (Priority: High): If walking calories are not supplied, the system shall calculate burn: Calories = Steps * (Weight in kg / 70) * 0.045.

- FR-ACT-03 (Priority: High): The system shall upsert the activity record in MongoDB under the compound unique index { userId: 1, date: -1 } ensuring exactly one consolidated entry per user per day.

- FR-ACT-04 (Priority: Medium): The system shall calculate consecutive activity streaks by evaluating historical dates in descending order.

3.2.3.4 Outputs

- HTTP 200 OK with the saved activity log document, distance, calories, and calculated streak count.

- Paginated activity history array sorted by date descending.

3.2.3.5 Error Handling

- Return HTTP 400 Bad Request if date is not formatted as YYYY-MM-DD or step count is negative.

- Return HTTP 404 Not Found if attempting to edit or delete a non-existent activity record.

3.2.4 Nutrition and Caloric Accounting Subsystem

3.2.4.1 Introduction

Manages food catalog exploration, custom food item creation, itemized meal logging, and daily macronutrient summation.

3.2.4.2 Inputs

- Meal Log: Date (YYYY-MM-DD), mealType (enum: breakfast, lunch, dinner, snack, drink), items array (each containing foodItemId, name, servingSize, servingUnit, quantity, calories, protein, carbs, fat, fiber).

- Food Catalog Search: Text query string (q) and optional category filter.

- Custom Food Item: Name, category, servingSize, servingUnit, calories, protein, carbs, fat, fiber.

3.2.4.3 Processing

- FR-NUT-01 (Priority: High): The system shall provide full-text search across catalog food items using MongoDB text index on name and category, falling back to case-insensitive regular expressions for partial matches.

- FR-NUT-02 (Priority: High): The system shall support public catalog items (including 45+ Indian traditional foods) and private user-created foods.

- FR-NUT-03 (Priority: High): When logging a meal, the system shall calculate server-side totals: totalCalories = sum of (item.calories * item.quantity), and similarly for protein, carbs, fat, and fiber.

- FR-NUT-04 (Priority: High): The system shall embed the full macro snapshot within each item of the meal document, preserving historical integrity if the parent catalog item is modified later.

3.2.4.4 Outputs

- HTTP 200 OK with search results list of food items.

- HTTP 201 Created with saved meal document and verified macro totals.

- HTTP 200 OK with day's meal logs and daily nutritional totals.

3.2.4.5 Error Handling

- Return HTTP 400 Bad Request if meal items array is empty or numeric quantities are zero or negative.

- Return HTTP 404 Not Found if meal ID for deletion is not found or belongs to another user.

3.2.5 Exercise and Progressive Overload Tracking Subsystem

3.2.5.1 Introduction

Supports logging multi-set resistance exercises, cardiovascular workouts, training volume calculation, and workout template deployment.

3.2.5.2 Inputs

- Workout Log: Date (YYYY-MM-DD), workoutType (enum: strength_training, cardio, yoga, pilates, walking, running, cycling, mobility, custom), durationMinutes (integer, 1–1440), exercises array (exerciseName, sets array with setNumber, reps, weightKg, restTimeSeconds), optional notes.

- Workout Template: Name, category, exercises array with defaultSets, defaultReps, defaultWeightKg.

3.2.5.3 Processing

- FR-WRK-01 (Priority: High): If estimated calories burned is not provided, the system shall compute it using standard MET values: strength_training (5.0), cardio (8.0), running (9.8), cycling (7.5), walking (3.8), yoga (3.0), pilates (3.5), mobility (2.8), custom (5.0), applied via formula: Calories = (MET * 3.5 * weightKg / 200) * durationMinutes.

- FR-WRK-02 (Priority: High): The system shall compute training volume for resistance workouts as the sum of (reps * weightKg) across all sets of all exercises.

- FR-WRK-03 (Priority: Medium): The system shall store and retrieve reusable workout templates linked to the user ID.

3.2.5.4 Outputs

- HTTP 201 Created with saved workout log document including embedded exercises and sets.

- Paginated workout history and list of saved templates.

3.2.5.5 Error Handling

- Return HTTP 400 Bad Request if duration is less than 1 minute or sets contain negative values.

- Return HTTP 404 Not Found if target workout ID does not exist or unauthorized.

3.2.6 Sleep Architecture and Recovery Monitoring Subsystem

3.2.6.1 Introduction

Records sleep duration across midnight boundaries, tracks self-reported sleep quality, and computes schedule consistency.

3.2.6.2 Inputs

- sleepStart: ISO-8601 UTC timestamp string (e.g. 2026-10-14T23:30:00.000Z).

- sleepEnd: ISO-8601 UTC timestamp string (e.g. 2026-10-15T07:30:00.000Z).

- quality: Integer from 1 (poor) to 5 (excellent).

- Optional notes string.

3.2.6.3 Processing

- FR-SLP-01 (Priority: High): The system shall calculate durationMinutes as (sleepEnd - sleepStart) in milliseconds divided by 60,000, accurately supporting sessions that span across midnight.

- FR-SLP-02 (Priority: High): The system shall validate that sleepEnd is chronologically later than sleepStart and total duration does not exceed 1,440 minutes (24 hours).

- FR-SLP-03 (Priority: Medium): The system shall calculate statistical sleep consistency via MongoDB aggregation using population standard deviation ($stdDevPop) on duration minutes across evaluated nights.

3.2.6.4 Outputs

- HTTP 201 Created with saved sleep log document.

- Paginated sleep history and analytical summary (average hours, average quality, consistency index).

3.2.6.5 Error Handling

- Return HTTP 400 Bad Request if sleepEnd is before or equal to sleepStart or quality rating is outside the 1–5 range.

3.2.7 Hydration Tracking Subsystem

3.2.7.1 Introduction

Manages milliliter-level water consumption logging, quick-add shortcuts, daily hydration targets, and weekly trend analysis.

3.2.7.2 Inputs

- amountMl: Positive integer (1 to 5,000 ml).

- timestamp: Optional ISO timestamp (defaults to server current time).

3.2.7.3 Processing

- FR-HYD-01 (Priority: High): The system shall record discrete water intake events in the water_logs collection.

- FR-HYD-02 (Priority: High): The system shall aggregate all intake logs for a specified calendar day (00:00:00.000Z to 23:59:59.999Z) to compute the current daily intake total.

- FR-HYD-03 (Priority: Medium): The system shall generate a 7-day hydration trend grouping water intake by calendar day using $dateToString.

3.2.7.4 Outputs

- HTTP 201 Created with saved water log entry.

- HTTP 200 OK with daily intake total, list of individual logs, and 7-day trend array.

3.2.7.5 Error Handling

- Return HTTP 400 Bad Request if amountMl is less than 1 or exceeds 5,000 ml in a single entry.

3.2.8 Anthropometric Body Measurement Subsystem

3.2.8.1 Introduction

Facilitates tracking of body weight and body circumferences, automatically synchronizing current user baseline metrics.

3.2.8.2 Inputs

- date: String (YYYY-MM-DD).

- weightKg: Number (20 to 500 kg).

- Optional circumferences: waistCm, hipCm, chestCm, armCm, thighCm (each in realistic cm ranges).

- Optional progress notes.

3.2.8.3 Processing

- FR-MEA-01 (Priority: High): The system shall upsert body measurements under the compound index { userId: 1, date: -1 }.

- FR-MEA-02 (Priority: High): Upon recording a measurement, the system shall automatically update the currentWeight attribute on the user's root profile document.

- FR-MEA-03 (Priority: Medium): The system shall calculate net weight change as the difference between the most recent measurement and the oldest recorded baseline.

3.2.8.4 Outputs

- HTTP 201 Created with saved measurement record.

- HTTP 200 OK with measurement history and total weight progression delta.

3.2.8.5 Error Handling

- Return HTTP 400 Bad Request if weightKg is out of bounds or date format is invalid.

3.2.9 Fitness Goal Management Subsystem

3.2.9.1 Introduction

Maintains user-defined quantitative targets across seven health domains and calculates progress percentages.

3.2.9.2 Inputs

- goalType: Enum (steps, calories, protein, water, sleep, workouts, weight).

- targetValue: Positive number.

- unit: String (e.g. steps, kcal, g, ml, minutes, sessions/week, kg).

- Optional effectiveFrom, effectiveTo, isActive flag.

3.2.9.3 Processing

- FR-GOL-01 (Priority: High): When creating a new active goal of a given type, the system shall automatically deactivate previous active goals of the same type.

- FR-GOL-02 (Priority: High): The system shall calculate real-time progress for each active goal by cross-referencing current logs: steps from activity_logs, calories and protein from meallogs, water from water_logs, sleep from sleep_logs, workouts from workoutlogs, and weight from body_measurements.

- FR-GOL-03 (Priority: Medium): If step, calorie, water, or sleep goals are established, the system shall synchronize corresponding user preferences on the user profile.

3.2.9.4 Outputs

- HTTP 201 Created with saved goal document.

- HTTP 200 OK with active goals list containing currentProgress and percentage attributes.

3.2.9.5 Error Handling

- Return HTTP 400 Bad Request if targetValue is zero or negative or goalType is unrecognized.

3.2.10 Advanced Analytics and MongoDB Aggregation Engine Subsystem

3.2.10.1 Introduction

Executes database-level multi-stage aggregation pipelines for high-performance analytics without in-memory processing.

3.2.10.2 Inputs

- userId: Authenticated user ObjectId.

- Optional startDate and endDate query parameters.

- Optional evaluation window parameter (days; default 14).

3.2.10.3 Processing

- FR-AGG-01 (Priority: High - Pipeline 1): Execute weekly step aggregation grouping by calendar day with $sum, $round, and $sort, producing daily distance, calories, and summary stats ($max, $min, $avg).

- FR-AGG-02 (Priority: High - Pipeline 2): Execute nutrition aggregation grouping meal logs by date and mealType, unwinding items to sum calories and macronutrients.

- FR-AGG-03 (Priority: High - Pipeline 3): Execute workout volume analysis unwinding exercises and sets, multiplying reps by weight to rank exercises by total volume lifted.

- FR-AGG-04 (Priority: High - Pipeline 4): Execute sleep aggregation computing average duration, average quality, and duration dispersion ($stdDevPop) to generate schedule consistency scores.

- FR-AGG-05 (Priority: High - Pipeline 5): Execute calorie balance cross-collection join ($lookup) connecting activity_logs with meallogs and workoutlogs by composite key (userId + date), projecting net calorie balance.

- FR-AGG-06 (Priority: Medium - Pipeline 6): Execute monthly progress aggregation using $substr on date strings (YYYY-MM), grouping monthly steps, workout frequency, and net weight changes.

- FR-AGG-07 (Priority: Medium - Pipeline 7): Execute community activity leaderboard aggregation grouping step totals by userId, joining users via $lookup, and anonymizing display names (First Name + Last Initial).

- FR-AGG-08 (Priority: High - Pipeline 8): Compute the 5-pillar composite Lifestyle Consistency Score (0–100) weighting: Steps Adherence (25%), Workout Regularity (25%), Sleep Target Adherence (20%), Hydration Target Adherence (15%), and Nutrition Logging Consistency (15%). Include non-clinical habit disclaimer.

3.2.10.4 Outputs

- HTTP 200 OK containing JSON analytical structures corresponding to the requested aggregation pipeline.

3.2.10.5 Error Handling

- Return HTTP 401 Unauthorized if request is unauthenticated.

- Return HTTP 500 Internal Server Error if aggregation execution fails on database cluster.

3.5 Non-Functional Requirements

3.5.1 Performance

- NFR-PERF-01: 95% of standard CRUD API requests (activity logging, water addition, meal insertion) shall respond in under 200 milliseconds under normal operating loads.

- NFR-PERF-02: 95% of complex aggregation queries (Dashboard overview, Calorie Balance $lookup, Weekly Step Analysis) shall execute on the database tier in under 150 milliseconds for date ranges up to 30 days.

- NFR-PERF-03: The frontend bundle initial payload shall remain under 1 MB uncompressed and under 300 KB gzip-compressed.

- NFR-PERF-04: Client-side UI transitions and chart re-renders shall maintain a minimum rendering frame rate of 55 frames per second on standard hardware.

3.5.2 Reliability

- NFR-REL-01: Mean Time Between Failures (MTBF) shall exceed 720 continuous operating hours in a hosted production environment.

- NFR-REL-02: Zero data corruption or silent truncation shall occur on financial or biometric values; all numbers shall maintain schema precision via Mongoose validation.

- NFR-REL-03: The database seed utility shall be strictly idempotent; executing the seeder multiple times shall reset the demo user to exactly 30 days of clean logs without duplicate record generation.

3.5.3 Availability

- NFR-AVL-01: The system shall achieve 99.5% operational uptime during scheduled operational windows, excluding announced maintenance periods.

- NFR-AVL-02: The backend API shall provide a public health monitoring endpoint (GET /api/health) returning service status and UTC timestamp with under 50 millisecond response time.

3.5.4 Security

- NFR-SEC-01: Passwords shall be cryptographically hashed using bcrypt with salt rounds of 10. Passwords shall be marked with select: false in Mongoose models to prevent accidental query exposure.

- NFR-SEC-02: Authentication tokens shall be signed using HMAC SHA-256 with an environment-configured secret key of minimum 256 bits of entropy.

- NFR-SEC-03: API endpoints under /api/* shall be protected by an IP-based rate limiter restricting traffic to 500 requests per 15-minute window per IP.

- NFR-SEC-04: Security HTTP headers shall be enforced via Helmet middleware, disabling MIME sniffing, restricting framing to prevent clickjacking, and enforcing strict referrer policies.

- NFR-SEC-05: Multi-tenant user isolation shall be enforced at the middleware and controller levels. Database queries for private resources shall strictly filter by the authenticated user's ObjectId.

- NFR-SEC-06: Database query inputs shall be validated via Zod schemas to mitigate NoSQL query injection vectors.

3.5.5 Maintainability

- NFR-MNT-01: Source code shall be strictly typed using TypeScript across both frontend and backend configurations with zero compilation errors in strict mode.

- NFR-MNT-02: The codebase shall adhere to a modular architecture separating models, controllers, routes, middleware, validation schemas, services, and utilities.

- NFR-MNT-03: Automated test suites (Jest + Supertest) shall maintain 100% pass rates across core authentication, CRUD, and aggregation test suites.

3.5.6 Portability

- NFR-POR-01: The backend application shall execute consistently across Windows (PowerShell compatible), macOS, and Linux (Ubuntu 20.04+) environments without code modification.

- NFR-POR-02: The application shall support deployment on containerized and modern serverless/PaaS platforms (Vercel for frontend SPA, Render/Railway for backend Express, MongoDB Atlas for database tier).

3.7 Design Constraints

- DC-01 Framework Constraint: Implementation is strictly constrained to the MERN stack (MongoDB, Express.js, React, Node.js) implemented in TypeScript.

- DC-02 Aggregation Constraint: Analytic metrics displayed on dashboard and analytics pages must be calculated directly within MongoDB aggregation pipelines. Computing summary stats via JavaScript loops over raw document arrays in application memory is prohibited.

- DC-03 Free & Open Source Constraint: The system must run without requiring paid third-party API subscriptions or proprietary hardware sensor SDKs.

- DC-04 Non-Clinical Disclaimer Constraint: All calculated health indices (caloric balance, consistency score) must feature clear disclaimers that the application provides lifestyle habit tracking and not clinical or medical risk assessments.

- DC-05 Idempotent Seeding Constraint: A repeatable database seed script must be provided that generates a realistic 30-day historical dataset for demonstration purposes without manual entry.

3.9 Other Requirements

- OR-01 Regional Food Catalog: The system shall incorporate a starter food catalog containing at least 40 verified items, with explicit inclusion of traditional Indian staples (Roti, Steamed Rice, Moong Dal Tadka, Rajma Masala, Chole Masala, Palak Paneer, Curd, Upma, Poha, Idli, Dosa).

- OR-02 Single-Click Evaluation Demo: The login user interface shall provide a prominent 'Quick Fill Demo Account' button that populates pre-configured demo credentials (demo@fitpulse.app / Password123!) to streamline academic presentations and evaluator reviews.

- OR-03 Timezone Neutrality: All timestamps for sleep and water logs shall be stored internally as ISO-8601 UTC dates, with date-only records stored as YYYY-MM-DD strings to prevent timezone day-shifting errors.

4. Analysis Models

This section presents the formal analysis models employed to analyze and validate the specific requirements of FitPulse — Your Personal Fitness Intelligence Platform. These models describe functional interactions, data movement, system states, and conceptual document relationships. Each analysis model includes an introductory overview, a detailed narrative description, and an explicit traceability mapping back to the functional requirements specified in Section 3.

4.1 Use Case Model

4.1.1 Introduction

The Use Case Model delineates the functional interactions between external human actors and the FitPulse system boundary. It defines the primary system functions from an actor-oriented operational perspective, establishing the behavioral foundation for requirements verification.

4.1.2 Actors

1. Unauthenticated Visitor: An anonymous individual interacting with the system through a web client who can register a new account, log into an existing account, or access the pre-configured academic demonstration account.

2. Authenticated Athlete: A registered user possessing valid cryptographic authorization credentials (JWT) who inputs biometric data, manages personal fitness goals, reviews live dashboard visualizations, and configures account settings.

3. Database Processing Engine: The internal MongoDB database runtime responsible for executing multi-stage aggregation pipelines ($match, $group, $unwind, $lookup, $facet), evaluating indexing constraints, and enforcing document validation schemas.

4.1.3 Use Case Specifications

Use Case UC-01: User Authentication & Session Establishment

- Primary Actor: Unauthenticated Visitor

- Introduction: Manages new user onboarding, password hashing, credential verification, and issuance of signed session tokens.

- Narrative Description: The visitor inputs registration details (name, email, password, baseline height, weight, activity level, fitness objective) or existing login credentials. The server validates payloads against Zod schemas. Upon validation, the server hashes the password via bcrypt (salt rounds: 10) during registration or verifies the hash during login. A signed 7-day JWT is generated and transmitted via an HttpOnly cookie and JSON response body.

- Traceability: Traces to FR-AUTH-01, FR-AUTH-02, FR-AUTH-03, FR-AUTH-04, NFR-SEC-01, NFR-SEC-02.

Use Case UC-02: Daily Step Logging & Pedometer Estimation

- Primary Actor: Authenticated Athlete

- Introduction: Enables recording of daily step counts with automatic stride-based distance and weight-based caloric expenditure calculation.

- Narrative Description: The athlete navigates to the Activity interface and enters a calendar date (YYYY-MM-DD) and step count. If the athlete does not supply distance or calories burned, the system evaluates the athlete's recorded height to calculate stride length (Height * 0.414 / 100 meters) and evaluates body weight to calculate walking calories (Steps * [Weight / 70] * 0.045). The entry is upserted into MongoDB under a compound unique index (userId + date), maintaining exactly one record per day. The system iterates through consecutive prior days to compute active streaks.

- Traceability: Traces to FR-ACT-01, FR-ACT-02, FR-ACT-03, FR-ACT-04, NFR-PERF-01.

Use Case UC-03: Itemized Meal Logging & Macro Accounting

- Primary Actor: Authenticated Athlete

- Introduction: Facilitates searching the food catalog, creating custom food items, and logging itemized meals with immutable macro snapshots.

- Narrative Description: The athlete selects a meal category (Breakfast, Lunch, Dinner, Snack, Drink) and date. The athlete queries the food catalog, which performs text-indexed searches across 45+ pre-loaded items (including Indian staples such as Roti, Dal, Paneer, Rice, and Chole). The athlete adjusts portion quantities. The server computes total meal calories and macronutrient weights (protein, carbohydrates, fats, fiber), embeds the exact nutritional snapshot inside the meal item array, and saves the document to MongoDB.

- Traceability: Traces to FR-NUT-01, FR-NUT-02, FR-NUT-03, FR-NUT-04, OR-01.

Use Case UC-04: Resistance & Cardiovascular Workout Logging

- Primary Actor: Authenticated Athlete

- Introduction: Supports structured exercise tracking with set-by-set resistance logging, progressive overload volume calculation, and MET-based caloric expenditure modeling.

- Narrative Description: The athlete enters workout parameters (type, duration, notes) or applies a pre-saved workout routine template. For resistance exercises, the athlete adds sets

containing repetition counts and weight lifted (kg). The server computes total training volume (Sum of reps * weight) and calculates estimated caloric expenditure using standard MET physiological constants scaled by duration and user body weight.

- Traceability: Traces to FR-WRK-01, FR-WRK-02, FR-WRK-03, NFR-PERF-01.

Use Case UC-05: Sleep Tracking & Midnight Boundary Resolution

- Primary Actor: Authenticated Athlete

- Introduction: Accurately records sleep intervals spanning across midnight boundaries and tracks qualitative recovery ratings.

- Narrative Description: The athlete enters sleep start (bedtime) and sleep end (wake-up) timestamps via datetime-local inputs, accompanied by a 1–5 star self-reported quality rating and optional notes. The server converts inputs to ISO-8601 UTC dates, calculates duration in minutes across midnight, validates that duration is positive and under 24 hours, and persists the entry into MongoDB.

- Traceability: Traces to FR-SLP-01, FR-SLP-02, FR-SLP-03, OR-03.

Use Case UC-06: Hydration Logging & Target Monitoring

- Primary Actor: Authenticated Athlete

- Introduction: Captures granular milliliter water intake through single-tap presets or custom volume entries.

- Narrative Description: The athlete taps a quick-add increment (+100 ml, +250 ml, +500 ml) or submits a custom milliliter volume. The server records the entry in the water logs collection. The system aggregates all entries matching the selected calendar day (00:00:00.000Z to 23:59:59.999Z), evaluates the total against the user's daily hydration target, and returns the updated daily volume and percentage to animate the circular progress ring.

- Traceability: Traces to FR-HYD-01, FR-HYD-02, FR-HYD-03, NFR-PERF-01.

Use Case UC-07: Body Composition & Anthropometric Progress Tracking

- Primary Actor: Authenticated Athlete

- Introduction: Records body weight and circumferences, synchronizing baseline user profile metrics.

- Narrative Description: The athlete records body weight in kg and optional circumferences (waist, hip, chest, arm, thigh) for a calendar date. The server upserts the entry under the user's measurement history, synchronizes the currentWeight attribute on the user's root profile document, and calculates net progression relative to the user's initial recorded baseline.

- Traceability: Traces to FR-MEA-01, FR-MEA-02, FR-MEA-03, NFR-PERF-01.

Use Case UC-08: Multi-Domain Fitness Goal Configuration

- Primary Actor: Authenticated Athlete

- Introduction: Establishes quantitative benchmarks across lifestyle domains and evaluates real-time achievement.

- Narrative Description: The athlete selects a goal domain (steps, calories, protein, water, sleep, workouts, weight), specifies a target value and unit, and submits the target. The system deactivates prior active goals of that type, persists the new goal, and synchronizes profile preferences where applicable. Active goals dynamically cross-reference live collection logs to display current completion percentages.

- Traceability: Traces to FR-GOL-01, FR-GOL-02, FR-GOL-03.

Use Case UC-09: Multi-Stage Analytical Aggregation & Habit Scoring

- Primary Actor: Authenticated Athlete, Database Processing Engine

- Introduction: Executes database-level data reductions across raw collections to deliver analytical charts and the composite Lifestyle Consistency Score.

- Narrative Description: When the dashboard or analytics page loads, the system dispatches requests to aggregation endpoints. The MongoDB engine processes multi-stage pipelines: grouping steps by calendar day, unwinding meal items to calculate macro distribution, calculating training volume, performing cross-collection joins ($lookup) between activity, meals, and workouts for net calorie balances, evaluating sleep duration variance ($stdDevPop), and computing the 5-pillar composite Lifestyle Consistency Score (0–100). The aggregated result is returned to the client for rendering via Recharts.

- Traceability: Traces to FR-DASH-01 through 08, FR-AGG-01 through 08, NFR-PERF-02, DC-02.

Use Case UC-10: Account Profile Modification & Cascading Data Deletion

- Primary Actor: Authenticated Athlete

- Introduction: Manages profile parameters, credential updates, and irreversible account termination.

- Narrative Description: The athlete updates profile parameters (name, height, baseline weight, activity level, measurement units) or submits a password update requiring current password verification. If the athlete opts to delete the account, a confirmation modal is presented. Upon confirmation, the server executes cascading deletions across all ten collections linked to the athlete's ObjectId, invalidates the authentication cookie, and redirects the client to the login screen.

- Traceability: Traces to FR-AUTH-05, NFR-SEC-05.

4.2 Data Flow Model

4.2.1 Introduction

The Data Flow Model maps the logical paths along which data moves through the FitPulse system, detailing how external inputs are sanitized, transformed, stored, and aggregated into analytical outputs.

4.2.2 Context Level Data Flow Diagram (Level 0) Narrative

- External Entity: User / Client Browser.

- Primary Process: FitPulse Information Management System.

- Data Inputs from User: Registration/login credentials, step logs, meal entries with item quantities, workout sets (reps and weights), sleep timestamps and quality scores, water intake entries, body measurements, and goal targets.

- Data Outputs to User: Authentication tokens and cookies, validation error notifications, live KPI dashboard metrics, interactive chart datasets (Recharts arrays), step history tables, nutritional summaries, training volume rankings, and composite consistency scores.

- External Entity: MongoDB Database Storage Engine (handling persistent BSON storage and pipeline query execution).

4.2.3 Level 1 Data Flow Diagram Narrative

Process 1.0 — Identity, Authentication & Access Control:

- Ingests raw registration or login credentials from the client.

- Evaluates inputs against RegisterSchema / LoginSchema via Zod.

- Interacts with Data Store D1 (Users) to verify email uniqueness or retrieve passwordHash.

- Executes bcrypt cryptographic verification.

- Outputs signed JWT token, sets HttpOnly cookie, and establishes user session context (req.user) for all downstream processes.

Process 2.0 — Biometric Ingestion & Schema Validation:

- Receives user inputs for steps, meals, workouts, sleep, water, measurements, and goals.

- Verifies format, data types, and numerical ranges using Zod validation middleware.

- Verifies that request identity matches authenticated user ID from Process 1.0.

- Routes sanitized payloads to respective business logic modules.

Process 3.0 — Biometric Processing & Algorithmic Calculation:

- Pedometer Logic: Applies height-proportional stride formula to calculate distance in km; applies body weight scaling to calculate walking calories.

- Nutritional Logic: Computes item-level calories and macro sums; embeds immutable macro snapshots.

- Workout Logic: Applies MET physiological constants to compute workout calories; multiplies repetitions by weight to calculate set volume.

- Sleep Logic: Evaluates ISO timestamps across midnight boundaries to calculate durationMinutes.

- Persists processed documents into respective Data Stores: D2 (Activity Logs), D3 (Meal Logs & Food Catalog), D4 (Workout Logs & Templates), D5 (Sleep Logs), D6 (Water Logs), D7 (Body Measurements), D8 (Goals).

Process 4.0 — Multi-Stage Database Aggregation Engine:

- Dispatches aggregation pipeline queries directly to the MongoDB Engine.

- Executes Pipeline 1 ($match, $group, $sort) over D2 to produce weekly step trends.

- Executes Pipeline 2 ($unwind, $group) over D3 to produce macro distributions.

- Executes Pipeline 3 ($unwind, $multiply, $group) over D4 to rank training volumes.

- Executes Pipeline 4 ($group, $stdDevPop) over D5 to evaluate sleep duration consistency.

- Executes Pipeline 5 ($lookup cross-collection join) connecting D2, D3, and D4 to project net calorie balance.

- Executes Pipeline 6 ($substr, $group) over D2, D4, and D7 to compute monthly progress summaries.

- Executes Pipeline 7 ($group, $lookup, $substrCP) over D2 and D1 to construct the anonymized step leaderboard.

- Executes Pipeline 8 (multi-domain weighted calculation) synthesizing steps, workouts, sleep, water, and meal adherence into the Lifestyle Consistency Score (0–100).

Process 5.0 — Presentation Transformation & Chart Formatting:

- Receives aggregated pipeline results from Process 4.0 and daily logs from Process 3.0.

- Structures data into visualization-ready JSON formats (Recharts bar, line, area, and pie data structures).

- Transmits completed payloads to the Client Web Application for rendering.

4.2.4 Traceability of Data Flow Model

- Process 1.0 traces to FR-AUTH-01 through 04, NFR-SEC-01 through 06.

- Process 2.0 traces to NFR-SEC-06, DC-01.

- Process 3.0 traces to FR-ACT-01, FR-ACT-02, FR-NUT-03, FR-WRK-01, FR-WRK-02, FR-SLP-01, FR-HYD-02, FR-MEA-02.

- Process 4.0 traces to FR-AGG-01 through 08, FR-DASH-01 through 08, DC-02.

- Process 5.0 traces to UI-01, UI-03, NFR-PERF-04.

4.3 State Transition Model

4.3.1 Introduction

The State Transition Model defines the dynamic behavior of the FitPulse system by modeling the lifecycle states of critical entities, including the user authentication session, daily activity tracking, and goal progression.

4.3.2 User Authentication Session Lifecycle

- State 1: Unauthenticated. The user client possesses no valid session token or cookie. Access to private routes (/activity, /nutrition, /workouts, /dashboard) is intercepted and redirected to /login.

- Transition Event: User submits valid registration or login credentials.

- State 2: Authenticated & Active. The user client holds a valid, cryptographically verified JWT. Access to all functional trackers, dashboards, and profile settings is authorized. Every outgoing request includes the token.

- Transition Event: User remains active; token approaches expiration or user explicitly triggers logout.

- State 3: Logged Out. The user executes logout. The server clears the HttpOnly cookie, client-side localStorage tokens are erased, and the client transitions to State 1 (Unauthenticated).

- Transition Event: 7-day token expiration window elapses without renewal.

- State 4: Session Expired. API returns HTTP 401 Unauthorized. The client interceptor automatically removes stored tokens and redirects the user to the login screen.

- Transition Event: User requests account deletion.

- State 5: Terminated. The server cascades deletion across all user data stores. The user document and all associated logs are permanently expunged.

4.3.3 Daily Activity Adherence Lifecycle

- State 1: Unlogged Day. No activity record exists in the database for the given calendar date. The dashboard steps KPI displays 0 steps and 0% completion.

- Transition Event: User logs initial steps for the day (e.g., 4,500 steps against a 10,000 target).

- State 2: Incomplete Adherence. The activity document is saved with steps less than dailyStepGoal. Dashboard displays partial progress ring completion. The streak continuity check verifies whether the previous day was logged.

- Transition Event: User logs additional steps bringing total to or above dailyStepGoal (e.g., 10,200 steps).

- State 3: Target Achieved. Step count meets or exceeds dailyStepGoal. The dashboard circular progress displays 100% completion with lime highlight.

- Transition Event: The next calendar day arrives without step entry.

- State 4: Streak Broken. The active streak counter resets to 0 if a full calendar day is skipped without any logged activity.

4.3.4 Goal Progression Lifecycle

- State 1: Active In Progress. A goal is created with isActive set to true. Target value is established. Current progress is less than 100%.

- Transition Event: User biometric entries accumulate, meeting or exceeding the target value.

- State 2: Target Achieved. Progress reaches or exceeds 100%. The UI updates the goal card badge to 'Target Reached'.

- Transition Event: User defines a new target of the same goalType.

- State 3: Superseded / Inactive. The system sets isActive to false and timestamps effectiveTo with the current date, preserving historical performance while activating the new target.

4.3.5 Traceability of State Transition Model

- Authentication session states trace to FR-AUTH-01 through 05, NFR-SEC-01, NFR-SEC-02.

- Activity adherence states trace to FR-ACT-01 through 04, FR-DASH-02.

- Goal progression states trace to FR-GOL-01 through 03.

4.4 Entity Relationship & Document Analysis Model

4.4.1 Introduction

The Entity Relationship and Document Analysis Model specifies the conceptual document architecture, entity associations, and modeling strategies governing the ten MongoDB collections.

4.4.2 Embedded versus Referenced Document Architecture

FitPulse employs a hybrid document design combining the strengths of relational integrity and document-oriented nesting:

1. Referenced Architecture (ObjectId References):

- User to Biometric Logs: A one-to-many relationship exists between User and all operational log collections (ActivityLog, MealLog, WorkoutLog, SleepLog, WaterLog, BodyMeasurement, Goal, WorkoutTemplate). Each log document references the parent User via an indexed userId (Schema.Types.ObjectId, ref: 'User'). This avoids unbounded document growth, supports infinite chronological logging, and simplifies safe pagination.

- Food Catalog Reference: MealLog items store an optional foodItemId reference to the catalog document in FoodItem, while also embedding a full nutritional snapshot.

2. Embedded Document Architecture:

- Workout Exercises & Sets: ExerciseEntry documents and their child WorkoutSet documents are strictly embedded within the WorkoutLog document. Workout sets exist exclusively within the context of a specific session and are always queried together, eliminating expensive relational multi-table joins.

- Meal Items & Nutritional Snapshots: MealItem documents are embedded directly within the MealLog document. Each item stores its own calculated calories, protein, carbs, fat, and fiber based on the portion quantity at log time. This isolates historical meal logs from future modifications or deletions in the master food catalog.

- User Preferences: Preferences (measurementUnits, dailyStepGoal, dailyCalorieGoal, dailyWaterGoal, dailySleepGoal) are embedded directly within the User document for single-read retrieval during session authentication.

4.4.3 Entity Relationship Structural Summary

- User (1) to ActivityLog (0..*): Key: userId. Relationship: One user creates many daily activity records. Unique constraint on (userId, date).

- User (1) to MealLog (0..*): Key: userId. Relationship: One user creates many meal logs across calendar dates. Compound index on (userId, date).

- User (1) to WorkoutLog (0..*): Key: userId. Relationship: One user creates many workout sessions embedding exercises and sets. Compound index on (userId, date).

- User (1) to SleepLog (0..*): Key: userId. Relationship: One user creates many sleep interval records. Compound index on (userId, sleepStart).

- User (1) to WaterLog (0..*): Key: userId. Relationship: One user creates many hydration entries. Compound index on (userId, timestamp).

- User (1) to BodyMeasurement (0..*): Key: userId. Relationship: One user logs many anthropometric check-ins. Compound index on (userId, date).

- User (1) to Goal (0..*): Key: userId. Relationship: One user defines many active and historical targets. Compound index on (userId, goalType, isActive).

- User (1) to WorkoutTemplate (0..*): Key: userId. Relationship: One user saves many reusable routine templates.

- FoodItem (1) to MealItem (0..*): Key: foodItemId. Relationship: Optional catalog reference. The embedded MealItem preserves an independent immutable copy of macro values.

4.4.4 Traceability of Document Analysis Model

- Embedded workout sets trace to FR-WRK-01, FR-WRK-02.

- Embedded meal macro snapshots trace to FR-NUT-03, FR-NUT-04.

- Referenced user collections trace to FR-AUTH-05, NFR-SEC-05, DC-01.

- Document indexing structures trace to NFR-PERF-01, NFR-PERF-02.

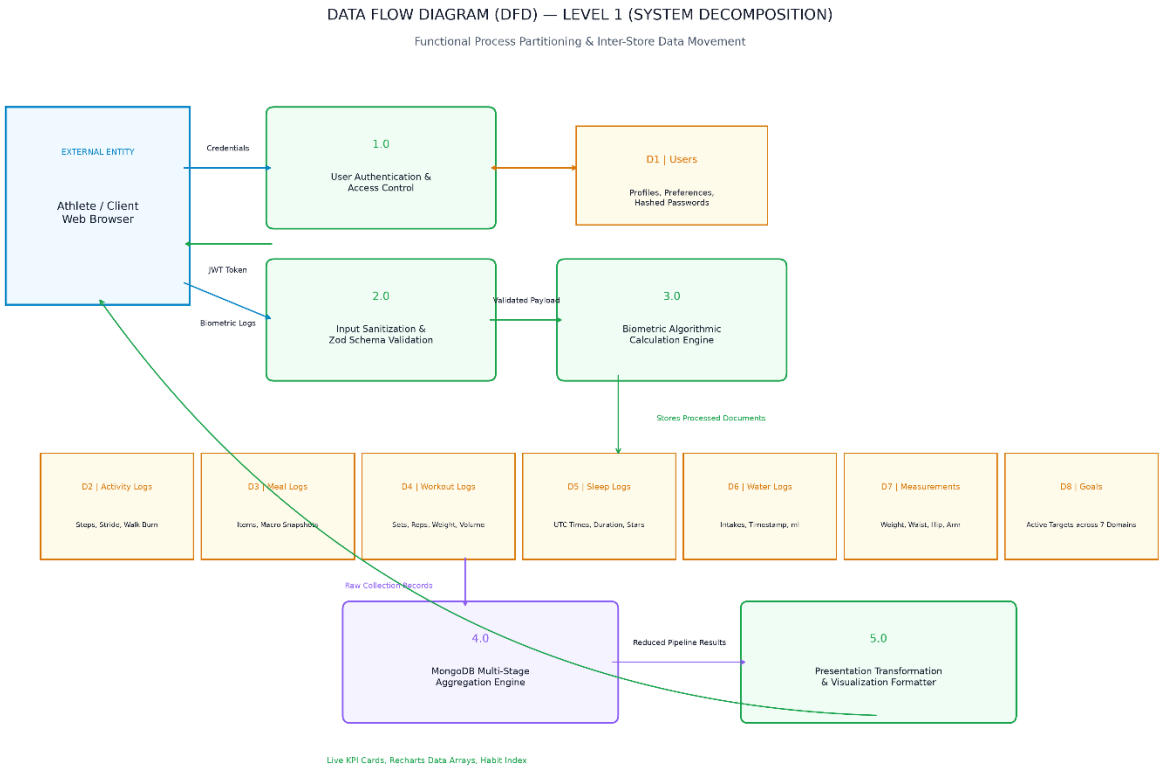


Figure 4.2: DFD Level 1 — System-Wide Functional Decomposition of FitPulse

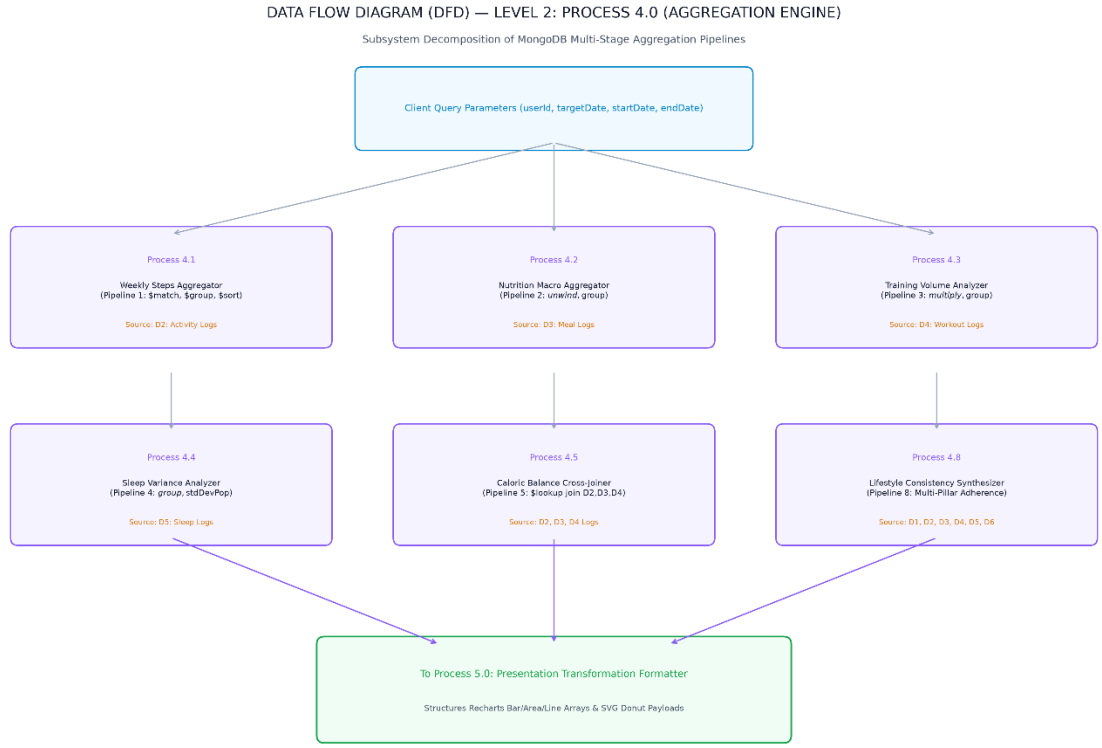


Figure 4.3: DFD Level 2 — Subsystem Decomposition of Process 4.0 (MongoDB Aggregation Engine)

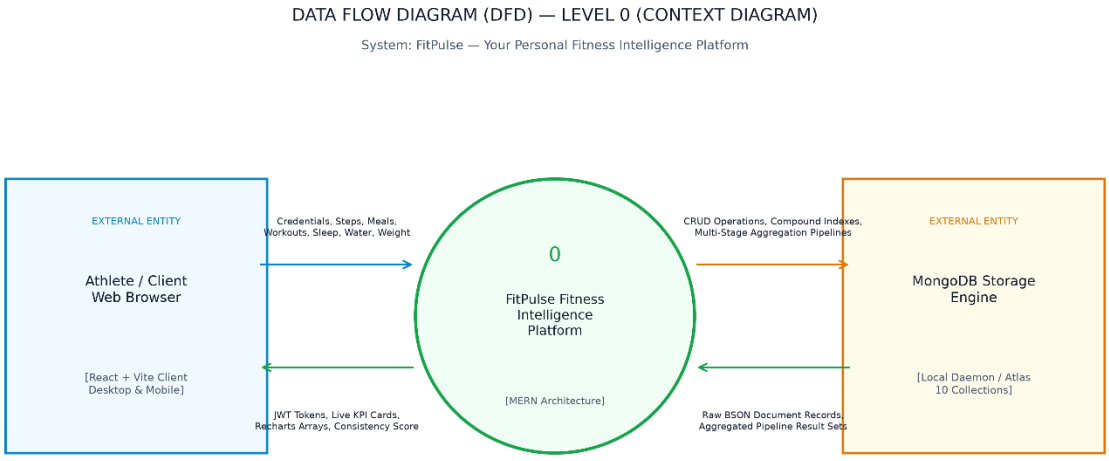


Figure 4.1: DFD Level 0 — Context Diagram of FitPulse Intelligence Platform

A. Appendices

This section provides supplementary technical, mathematical, and data catalog information to accompany the Software Requirements Specification for FitPulse — Your Personal Fitness Intelligence Platform.

Status of Appendices with Respect to Requirements:

- Appendix A.1 (MongoDB Database Collection Schemas & Indexing Definitions) and Appendix A.2 (Starter Food Catalog & Regional Indian Nutrition Composition Table) contain normative technical specifications that are formally considered part of the binding requirements of this SRS document.

- Appendix A.3 (Physical Activity MET Compendium & Biometric Calculation Formulas) and Appendix A.4 (Academic Demonstration Walkthrough Script & Pre-Configured Test Credentials) contain informative reference material intended to assist developers, evaluators, and academic examiners during review and demonstration.

A.1 Appendix 1 — MongoDB Database Collection Schemas, Field Types & Index Specifications (Normative)

This appendix formally details the ten collection schemas, field data types, nullability, validation rules, and indexing structures enforced by Mongoose ODM within MongoDB.

Collection 1: users

- _id: Schema.Types.ObjectId (Primary key, auto-generated)

- name: String, required, trim, min length 2, max length 100

- email: String, required, unique, lowercase, trim, RFC 5322 regex match. Single-field unique index: { email: 1 }

- passwordHash: String, required, select: false (suppressed from standard queries to prevent exposure)

- dateOfBirth: Date, optional

- height: Number, min 50, max 300 (centimeters)

- currentWeight: Number, min 20, max 500 (kilograms)

- activityLevel: String enum ('sedentary', 'lightly_active', 'moderately_active', 'very_active', 'extra_active'), default 'moderately_active'

- fitnessGoal: String enum ('muscle_gain', 'fat_loss', 'weight_maintenance', 'general_fitness', 'improved_endurance'), default 'general_fitness'

- timezone: String, default 'UTC'

- preferences.measurementUnits: String enum ('metric', 'imperial'), default 'metric'

- preferences.dailyStepGoal: Number, min 1000, max 100000, default 10000

- preferences.dailyCalorieGoal: Number, min 800, max 10000, default 2200

- preferences.dailyWaterGoal: Number, min 500, max 10000, default 2500 (milliliters)

- preferences.dailySleepGoal: Number, min 180, max 900, default 480 (minutes / 8 hours)

- createdAt: Date, timestamps: true

- updatedAt: Date, timestamps: true

Collection 2: activity_logs

- _id: Schema.Types.ObjectId (Primary key)

- userId: Schema.Types.ObjectId, ref 'User', required, indexed

- date: String, format 'YYYY-MM-DD', required, indexed

- steps: Number, min 0, max 200000, required

- distanceKm: Number, min 0, required

- estimatedCaloriesBurned: Number, min 0, required

- source: String enum ('manual', 'sync', 'wearable'), default 'manual'

- createdAt: Date, timestamps: true

- Compound Index: { userId: 1, date: -1 }, unique: true (enforces exactly one record per user per day)

Collection 3: food_items

- _id: Schema.Types.ObjectId (Primary key)

- name: String, required, trim

- category: String, required, trim

- servingSize: Number, min 0.1, required

- servingUnit: String, required, trim

- calories: Number, min 0, required

- protein: Number, min 0, required

- carbs: Number, min 0, required

- fat: Number, min 0, required

- fiber: Number, min 0, default 0

- createdBy: Schema.Types.ObjectId, ref 'User', default null (null represents system-wide catalog items)

- isPublic: Boolean, default true

- Text Index: { name: 'text', category: 'text' } (powers full-text catalog search)

Collection 4: meallogs

- _id: Schema.Types.ObjectId (Primary key)

- userId: Schema.Types.ObjectId, ref 'User', required, indexed

- date: String, format 'YYYY-MM-DD', required, indexed

- mealType: String enum ('breakfast', 'lunch', 'dinner', 'snack', 'drink'), required

- items: Array of embedded MealItem schemas:

  * foodItemId: Schema.Types.ObjectId, ref 'FoodItem', optional

  * name: String, required

  * servingSize: Number, required

  * servingUnit: String, required

  * quantity: Number, min 0.01, required

  * calories: Number, min 0, required

  * protein: Number, min 0, required

  * carbs: Number, min 0, required

  * fat: Number, min 0, required

  * fiber: Number, min 0, default 0

- totalCalories: Number, min 0, required

- totalProtein: Number, min 0, required

- totalCarbs: Number, min 0, required

- totalFat: Number, min 0, required

- totalFiber: Number, min 0, default 0

- Compound Indexes: { userId: 1, date: -1 }, { userId: 1, mealType: 1, date: -1 }

Collection 5: workoutlogs

- _id: Schema.Types.ObjectId (Primary key)

- userId: Schema.Types.ObjectId, ref 'User', required, indexed

- date: String, format 'YYYY-MM-DD', required, indexed

- workoutType: String enum ('strength_training', 'cardio', 'yoga', 'pilates', 'walking', 'running', 'cycling', 'mobility', 'custom'), required

- durationMinutes: Number, min 1, max 1440, required

- exercises: Array of embedded ExerciseEntry schemas:

  * exerciseName: String, required, trim

  * sets: Array of embedded WorkoutSet schemas:

    - setNumber: Number, min 1, required

    - reps: Number, min 0, required

    - weightKg: Number, min 0, default 0, required

    - restTimeSeconds: Number, min 0, default 60

  * notes: String, trim

- estimatedCaloriesBurned: Number, min 0, required

- notes: String, trim

- Compound Indexes: { userId: 1, date: -1 }, { userId: 1, workoutType: 1 }

Collection 6: sleep_logs

- _id: Schema.Types.ObjectId (Primary key)

- userId: Schema.Types.ObjectId, ref 'User', required, indexed

- sleepStart: Date, required, indexed (ISO UTC timestamp)

- sleepEnd: Date, required (ISO UTC timestamp)

- durationMinutes: Number, min 1, max 1440, required

- quality: Number, min 1, max 5, required (integer star rating)

- notes: String, trim

- Compound Index: { userId: 1, sleepStart: -1 }

Collection 7: water_logs

- _id: Schema.Types.ObjectId (Primary key)

- userId: Schema.Types.ObjectId, ref 'User', required, indexed

- timestamp: Date, default Date.now, indexed (ISO UTC timestamp)

- amountMl: Number, min 1, max 5000, required

- Compound Index: { userId: 1, timestamp: -1 }

Collection 8: body_measurements

- _id: Schema.Types.ObjectId (Primary key)

- userId: Schema.Types.ObjectId, ref 'User', required, indexed

- date: String, format 'YYYY-MM-DD', required, indexed

- weightKg: Number, min 20, max 500, required

- waistCm: Number, min 20, max 250, optional

- hipCm: Number, min 20, max 250, optional

- chestCm: Number, min 20, max 250, optional

- armCm: Number, min 10, max 100, optional

- thighCm: Number, min 15, max 150, optional

- notes: String, trim

- Compound Index: { userId: 1, date: -1 }

Collection 9: goals

- _id: Schema.Types.ObjectId (Primary key)

- userId: Schema.Types.ObjectId, ref 'User', required, indexed

- goalType: String enum ('steps', 'calories', 'protein', 'water', 'sleep', 'workouts', 'weight'), required

- targetValue: Number, min 0.1, required

- unit: String, required, trim

- effectiveFrom: Date, default Date.now

- effectiveTo: Date, optional

- isActive: Boolean, default true, indexed

- Compound Index: { userId: 1, goalType: 1, isActive: -1 }

Collection 10: workout_templates

- _id: Schema.Types.ObjectId (Primary key)

- userId: Schema.Types.ObjectId, ref 'User', required, indexed

- name: String, required, trim

- category: String, default 'strength_training', required

- exercises: Array of embedded TemplateExercise schemas:

  * exerciseName: String, required, trim

  * defaultSets: Number, min 1, default 3, required

  * defaultReps: Number, min 1, default 10, required

  * defaultWeightKg: Number, min 0, default 0, required

- Compound Index: { userId: 1, name: 1 }

A.2 Appendix 2 — Starter Food Catalog & Regional Nutrition Composition Table (Normative)

The FitPulse starter database includes verified nutritional profiles for 45+ items, with regional representation of common Indian staples, based on reference data from the Indian Council of Medical Research - National Institute of Nutrition (ICMR-NIN). All values represent average nutritional values per standard single serving.

Food Item Name | Category | Serving Size | Serving Unit | Calories (kcal) | Protein (g) | Carbs (g) | Fat (g) | Fiber (g)

Roti (Whole Wheat Chapati) | Indian Traditional | 1 | piece (40g) | 104 | 3.1 | 20.2 | 0.9 | 2.8

Steamed White Rice | Grains | 1 | bowl (150g) | 195 | 4.1 | 43.5 | 0.4 | 0.6

Brown Rice Cooked | Grains | 1 | bowl (150g) | 168 | 3.8 | 35.2 | 1.4 | 2.7

Rolled Oats (Cooked) | Grains | 1 | cup (234g) | 158 | 6.0 | 27.0 | 3.2 | 4.0

Idli (Steamed Rice & Lentil) | Indian Traditional | 2 | pieces (100g) | 140 | 4.2 | 28.0 | 0.6 | 1.8

Plain Dosa | Indian Traditional | 1 | piece (100g) | 168 | 3.9 | 29.2 | 3.7 | 1.4

Poha (Flattened Rice with Veggies) | Indian Traditional | 1 | bowl (180g) | 250 | 4.5 | 42.0 | 6.8 | 3.2

Upma (Semolina & Vegetables) | Indian Traditional | 1 | bowl (180g) | 230 | 5.2 | 38.0 | 6.0 | 2.9

Yellow Moong Dal Tadka | Lentils & Legumes | 1 | bowl (150g) | 150 | 9.2 | 19.8 | 3.8 | 4.5

Rajma Masala (Red Kidney Beans) | Lentils & Legumes | 1 | bowl (180g) | 215 | 11.5 | 31.0 | 4.8 | 8.2

Chole Masala (Chickpeas Curry) | Lentils & Legumes | 1 | bowl (180g) | 240 | 12.0 | 34.0 | 6.2 | 9.0

Black Dal (Dal Makhani) | Lentils & Legumes | 1 | bowl (180g) | 280 | 10.8 | 26.5 | 14.0 | 6.5

Boiled Chickpeas (Salad) | Lentils & Legumes | 1 | cup (164g) | 269 | 14.5 | 45.0 | 4.2 | 12.5

Sprouted Moong Salad | Lentils & Legumes | 1 | bowl (150g) | 135 | 10.5 | 22.0 | 0.8 | 6.2

Paneer (Raw Cottage Cheese) | Dairy | 100 | g | 265 | 18.3 | 3.4 | 20.8 | 0.0

Palak Paneer | Indian Traditional | 1 | bowl (200g) | 235 | 12.0 | 8.5 | 16.5 | 4.2

Tofu (Firm) | Proteins | 100 | g | 83 | 10.0 | 1.5 | 4.8 | 1.0

Curd / Plain Yogurt (Dahi) | Dairy | 1 | bowl (150g) | 98 | 5.3 | 7.0 | 5.0 | 0.0

Greek Yogurt (Non-fat) | Dairy | 1 | cup (170g) | 100 | 17.0 | 6.0 | 0.7 | 0.0

Cow Milk (Toned) | Dairy | 1 | glass (250ml) | 150 | 8.0 | 12.0 | 7.5 | 0.0

Soy Milk (Unsweetened) | Dairy | 1 | glass (250ml) | 80 | 7.0 | 4.0 | 4.0 | 2.0

Whey Protein Isolate (Vanilla) | Proteins | 1 | scoop (30g) | 120 | 25.0 | 2.0 | 1.0 | 0.0

Boiled Egg (Large) | Proteins | 1 | egg (50g) | 78 | 6.3 | 0.6 | 5.3 | 0.0

Egg White Omelette (3 whites) | Proteins | 1 | serving (100g) | 85 | 13.5 | 1.2 | 2.5 | 0.0

Grilled Chicken Breast | Proteins | 100 | g | 165 | 31.0 | 0.0 | 3.6 | 0.0

Mixed Vegetable Sabzi | Vegetables | 1 | bowl (150g) | 120 | 3.0 | 14.0 | 5.5 | 4.0

Bhindi Masala (Okra) | Vegetables | 1 | bowl (150g) | 110 | 2.5 | 11.0 | 6.0 | 4.5

Aloo Gobi | Vegetables | 1 | bowl (150g) | 140 | 3.2 | 18.0 | 6.2 | 3.8

Steamed Broccoli | Vegetables | 1 | cup (91g) | 31 | 2.5 | 6.0 | 0.4 | 2.4

Cucumber & Tomato Salad | Vegetables | 1 | plate (150g) | 45 | 1.5 | 8.0 | 0.5 | 2.5

Boiled Sweet Potato | Vegetables | 1 | medium (130g) | 112 | 2.0 | 26.0 | 0.1 | 3.9

Banana | Fruits | 1 | medium (118g) | 105 | 1.3 | 27.0 | 0.3 | 3.1

Apple (with skin) | Fruits | 1 | medium (182g) | 95 | 0.5 | 25.0 | 0.3 | 4.4

Orange | Fruits | 1 | medium (131g) | 62 | 1.2 | 15.4 | 0.2 | 3.1

Papaya Cubes | Fruits | 1 | cup (145g) | 62 | 0.7 | 15.8 | 0.4 | 2.5

Pomegranate Seeds | Fruits | 1 | cup (174g) | 144 | 3.0 | 32.7 | 2.0 | 7.0

Raw Almonds | Nuts & Seeds | 1 | handful (28g) | 164 | 6.0 | 6.1 | 14.2 | 3.5

Walnuts | Nuts & Seeds | 1 | handful (28g) | 185 | 4.3 | 3.9 | 18.5 | 1.9

Chia Seeds | Nuts & Seeds | 1 | tbsp (12g) | 58 | 2.0 | 5.0 | 3.7 | 4.1

Natural Peanut Butter | Nuts & Seeds | 2 | tbsp (32g) | 188 | 8.0 | 7.0 | 16.0 | 2.0

Green Tea (Unsweetened) | Beverages | 1 | cup (240ml) | 2 | 0.2 | 0.0 | 0.0 | 0.0

Masala Chai (with milk & 1 tsp sugar) | Beverages | 1 | cup (150ml) | 85 | 2.8 | 12.0 | 3.0 | 0.0

Coconut Water | Beverages | 1 | glass (250ml) | 45 | 1.7 | 8.9 | 0.5 | 2.6

Roasted Makhana (Foxnuts) | Snacks | 1 | bowl (30g) | 110 | 3.2 | 22.0 | 0.5 | 2.2

Dark Chocolate (70% Cacao) | Snacks | 1 | square (20g) | 115 | 1.8 | 9.0 | 8.5 | 2.0

A.3 Appendix 3 — Physical Activity MET Compendium & Biometric Calculation Formulas (Informative)

This appendix documents the biometric and mathematical formulas implemented in backend/src/utils/calculations.ts and the AnalyticsService.

1. Walking Distance Modeling:

Distance in kilometers is derived from user height. The anthropometric constant for human stride length is approximately 41.4% of standing height:

Stride Length (meters) = (Height in cm * 0.414) / 100

Distance (km) = (Steps * Stride Length in meters) / 1000 = Steps * (Height in cm * 0.414) / 100,000

If user height is not recorded in the profile, the system applies the population average stride constant of 0.000762 km (approximately 0.762 meters per step).

2. Walking Caloric Burn Estimation:

Walking calorie expenditure is proportional to steps taken and scaled relative to a standard 70 kg adult baseline (0.045 kcal per step):

Walking Calories (kcal) = Steps * (Body Weight in kg / 70) * 0.045

3. Workout Caloric Expenditure via Metabolic Equivalent of Task (MET):

Calculated using the 2011 Compendium of Physical Activities formula:

Caloric Expenditure (kcal) = [MET * 3.5 * Body Weight in kg / 200] * Duration in minutes

Standard MET Reference Table by Workout Category:

- strength_training: 5.0 MET (vigorous resistance lifting)
- cardio: 8.0 MET (circuit / interval training)
- running: 9.8 MET (approximately 6 mph / 9.6 km/h)
- cycling: 7.5 MET (moderate effort, stationary or road)
- walking: 3.8 MET (brisk pace, 3.5 mph)
- yoga: 3.0 MET (hatha / power yoga flow)
- pilates: 3.5 MET (mat / reformer exercise)
- mobility: 2.8 MET (stretching, dynamic flexibility)
- custom: 5.0 MET (general moderate athletic baseline)

4. Sleep Duration Resolution Across Midnight:

Sleep start and wake-up times are parsed as ISO UTC timestamps:

Duration (minutes) = (Epoch Timestamp of sleepEnd - Epoch Timestamp of sleepStart) / (1000 * 60)

Because epoch timestamps increment monotonically, sessions commencing at 23:30 on Date N and ending at 07:30 on Date N+1 correctly calculate as:

(1,793,356,200,000 - 1,793,327,400,000) / 60,000 = 480 minutes (8.0 hours).

5. Sleep Consistency Index Calculation:

Evaluates the population standard deviation of duration minutes across evaluated nights using the MongoDB aggregation operator $stdDevPop:

Sleep Consistency Score = Maximum of [0, Round(100 - (stdDevDuration * 0.5))]

A lower standard deviation indicates high schedule regularity, resulting in a score approaching 100%.

6. Progressive Overload Resistance Volume Formula:

Volume is computed for each completed set in a resistance session:

Set Volume (kg) = Repetitions * Weight Lifted in kg

Total Workout Volume (kg) = Sum of (Set Volume for all sets across all exercises)

7. Lifestyle Consistency Score Formula (0 to 100):

Synthesizes habit compliance across five independent weighted domains over an evaluation window (default 14 days):

- Pillar 1 — Steps Adherence (Weight: 25%): Ratio of actual steps to dailyStepGoal, capped at 1.0. Score = Ratio * 25.

- Pillar 2 — Workout Regularity (Weight: 25%): Ratio of completed workout sessions to expected frequency (target: 3 sessions per 7 days). Score = Ratio * 25.

- Pillar 3 — Sleep Target Adherence (Weight: 20%): Evaluates actual sleep duration proximity to dailySleepGoal. Ratio = 1 - (|duration - target| / target). Score = Ratio * 20.

- Pillar 4 — Hydration Target Adherence (Weight: 15%): Ratio of daily water intake to dailyWaterGoal, capped at 1.0. Score = Ratio * 15.

- Pillar 5 — Nutrition Logging Consistency (Weight: 15%): Ratio of days with at least one logged meal to total evaluated days. Score = Ratio * 15.

Total Composite Score = Steps Score + Workout Score + Sleep Score + Hydration Score + Nutrition Score.

Disclaimer: The score represents an algorithmic habit adherence tracking index and does not represent a clinical or medical risk diagnosis.

A.4 Appendix 4 — Academic Demonstration Walkthrough Script & Pre-Configured Test Credentials (Informative)

This appendix outlines the recommended presentation flow for academic examiners, university professors, and technical evaluators.

1. Demonstration Account Credentials:

- Email: demo@fitpulse.app

- Password: Password123!

- Convenience Feature: On the web login screen (/login), the user can click 'Quick Fill Demo Account' to populate these credentials and sign in immediately.

2. Seed Data Scope:

Executing the seed script (npm run seed in the backend directory) populates an idempotent 30-day realistic historical dataset:

- 1 Demo user profile with baseline metrics (Height: 178 cm, Weight: 75.5 kg, Goal: Muscle Gain)

- 45 Catalog food items including traditional Indian staples

- 30 Daily activity logs (ranging from 7,500 to 14,000 steps per day)

- 120 Categorized meal logs across 30 days (Breakfast, Lunch, Snacks, Dinner)

- 20 Exercise sessions with multi-set weight entries

- 30 Sleep records crossing midnight boundaries with 1–5 quality ratings

- 180 Hydration intake events (500 ml increments across 30 days)

- 5 Longitudinal body measurement records demonstrating progressive weight change

- 7 Active fitness goals across all health domains

- 2 Pre-configured workout routine templates

3. Recommended Step-by-Step Evaluation Walkthrough:

- Step 1: Open the client in a browser at http://localhost:5173. Click 'Quick Fill Demo Account' and submit.

- Step 2: On the Dashboard (/), review the six live KPI cards. Highlight that all values originate from MongoDB aggregation queries rather than hardcoded mock data.

- Step 3: Use the calendar date picker in the dashboard header to select earlier dates (e.g. 7 or 14 days ago) and observe reactive chart updates.

- Step 4: Navigate to Activity (/activity). Record a new step entry (e.g. 12,000 steps). Observe automatic distance and calorie calculation. Verify that the entry updates the history table.

- Step 5: Navigate to Nutrition (/nutrition). Search the catalog for 'Paneer' or 'Roti'. Add items to a lunch entry with custom quantities. Save and observe server-side macro summation.

- Step 6: Navigate to Workouts (/workouts). Select the 'Full Body Strength & Power' template. Review exercise set logging and volume computation.

- Step 7: Navigate to Sleep (/sleep). Review the midnight-crossing sleep duration and consistency index.

- Step 8: Navigate to Hydration (/hydration). Tap '+250 ml' or '+500 ml' and observe the circular progress ring update.

- Step 9: Navigate to Advanced Analytics (/analytics). Click the 'Aggregation Pipelines' tab to explain the MongoDB pipeline architecture ($match, $group, $unwind, $lookup, $project). Switch to the 'Activity Leaderboard' tab to demonstrate multi-tenant data aggregation and name anonymization.

- Step 10: Run the automated test suite in the backend terminal (npm test). Demonstrate that all 16 integration tests pass with 100% success.